

Despliegue de un Modelo de Predicción de Retrasos en Entregas Logísticas mediante FastAPI y Docker

Azadobay Cutiopala J.¹, Maldonado de la Cruz W.¹ y Viteri Ayala K.¹

¹Universidad Internacional del Ecuador

Despliegue de Modelos IA & IA en la Nube - Grupo 7

Docente: Bravo Guzmán Juan Gabriel

Resumen

Este informe documenta el proceso de despliegue de un modelo de inteligencia artificial como servicio web, en el marco del componente práctico de la Semana 1 de la materia Despliegue de Modelos IA & IA en la Nube. Se simula el caso de una empresa de logística que requiere predecir la probabilidad de retraso en una entrega a partir de la distancia del envío, el tipo de vehículo asignado y la hora del día en que inicia la entrega. Se documenta la metodología aplicada (generación de datos, entrenamiento del modelo, exposición como API REST y contenerización con Docker), los resultados obtenidos en las pruebas de predicción, y un conjunto de mejoras aplicadas al entregable base para reforzar su reproducibilidad y robustez.

1. Introducción

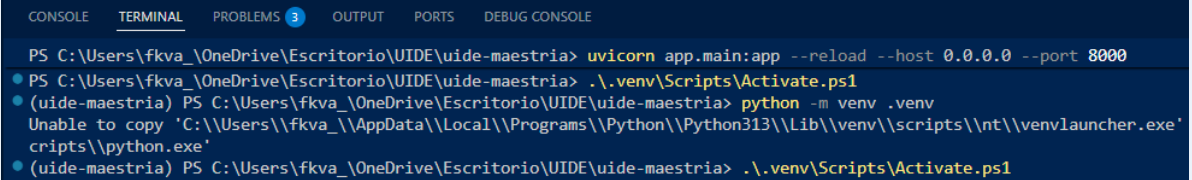
El despliegue de modelos de inteligencia artificial en entornos de producción constituye una etapa crítica del ciclo de vida de un proyecto de aprendizaje automático (MLOps), ya que un modelo entrenado solo genera valor real cuando es accesible, se puede mantener y es confiable dentro de una infraestructura operativa. El presente informe aborda este reto a través de un caso simulado de logística: una empresa que recibe eventos de entrega (distancia, tipo de vehículo y hora del día) y necesita anticipar si una entrega llegará tarde.

El objetivo del ejercicio es recorrer el flujo completo de despliegue, desde la generación de datos sintéticos hasta la exposición del modelo como un servicio REST contenerizado con Docker, aplicando los conceptos de arquitectura de sistemas de IA en producción, MLOps y estrategias de despliegue seguro revisados en la teoría de la semana.

2. Metodología

2.1. Preparación del entorno

Se creó un entorno virtual de Python (.venv) dentro de la carpeta del proyecto, aislado del resto del sistema, y se instalaron las dependencias necesarias mediante pip a partir del archivo requirements.txt: fastapi, uvicorn, pandas, scikit-learn y joblib.



```
CONSOLE  TERMINAL  PROBLEMS 3  OUTPUT  PORTS  DEBUG CONSOLE

PS C:\Users\fkva_\OneDrive\Escritorio\UIDE\uide-maestria> uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
• PS C:\Users\fkva_\OneDrive\Escritorio\UIDE\uide-maestria> .\.venv\Scripts\Activate.ps1
• (uide-maestria) PS C:\Users\fkva_\OneDrive\Escritorio\UIDE\uide-maestria> python -m venv .venv
Unable to copy 'C:\Users\fkva_\AppData\Local\Programs\Python\Python313\Lib\venv\scripts\nt\venvlauncher.exe'
cripts\python.exe'
• (uide-maestria) PS C:\Users\fkva_\OneDrive\Escritorio\UIDE\uide-maestria> .\.venv\Scripts\Activate.ps1
```

Figura 1. Creación y activación del entorno virtual.

```

(.venv) PS C:\Users\fkva_OneDrive\Escritorio\UIDE\uide-maestria\07-despliegue-modelos\Modelo-prediccion-entregas> pip install -r requirements.txt
Collecting fastapi (from -r requirements.txt (line 1))
  Using cached fastapi-0.141.1-py3-none-any.whl.metadata (27 kB)
Collecting uvicorn (from -r requirements.txt (line 2))
  Using cached uvicorn-0.52.1-py3-none-any.whl.metadata (6.6 kB)
Requirement already satisfied: pandas in C:\Users\fkva_OneDrive\Escritorio\UIDE\uide-maestria\.venv\Lib\site-packages (from -r requirements.txt (line 3))
Requirement already satisfied: scikit-learn in C:\Users\fkva_OneDrive\Escritorio\UIDE\uide-maestria\.venv\Lib\site-packages (from -r requirements.txt (line 4))
Requirement already satisfied: joblib in C:\Users\fkva_OneDrive\Escritorio\UIDE\uide-maestria\.venv\Lib\site-packages (from -r requirements.txt (line 5))
Collecting starlette>=0.46.0 (from fastapi->-r requirements.txt (line 1))
  Using cached starlette-1.6.0-py3-none-any.whl.metadata (6.4 kB)
Requirement already satisfied: pydantic>=2.9.0 in C:\Users\fkva_OneDrive\Escritorio\UIDE\uide-maestria\.venv\Lib\site-packages (from fastapi->-r requirements.txt (line 1))
Requirement already satisfied: typing-extensions>=4.8.0 in C:\Users\fkva_OneDrive\Escritorio\UIDE\uide-maestria\.venv\Lib\site-packages (from fastapi->-r requirements.txt (line 1))
Requirement already satisfied: typing-inspection>=0.4.2 in C:\Users\fkva_OneDrive\Escritorio\UIDE\uide-maestria\.venv\Lib\site-packages (from fastapi->-r requirements.txt (line 1))
Requirement already satisfied: annotated-doc>=0.0.2 in C:\Users\fkva_OneDrive\Escritorio\UIDE\uide-maestria\.venv\Lib\site-packages (from fastapi->-r requirements.txt (line 1))
Requirement already satisfied: click>=7.0 in C:\Users\fkva_OneDrive\Escritorio\UIDE\uide-maestria\.venv\Lib\site-packages (from uvicorn->-r requirements.txt (line 2))
Requirement already satisfied: h11>=0.8 in C:\Users\fkva_OneDrive\Escritorio\UIDE\uide-maestria\.venv\Lib\site-packages (from uvicorn->-r requirements.txt (line 2))
Requirement already satisfied: numpy>=1.26.0 in C:\Users\fkva_OneDrive\Escritorio\UIDE\uide-maestria\.venv\Lib\site-packages (from pandas->-r requirements.txt (line 3))
Requirement already satisfied: python-dateutil>=2.8.2 in C:\Users\fkva_OneDrive\Escritorio\UIDE\uide-maestria\.venv\Lib\site-packages (from pandas->-r requirements.txt (line 3))
Requirement already satisfied: tzdata in C:\Users\fkva_OneDrive\Escritorio\UIDE\uide-maestria\.venv\Lib\site-packages (from pandas->-r requirements.txt (line 3))
Requirement already satisfied: scipy>=1.10.0 in C:\Users\fkva_OneDrive\Escritorio\UIDE\uide-maestria\.venv\Lib\site-packages (from scikit-learn->-r requirements.txt (line 4))

```

Figura 2. Instalación de dependencias del proyecto.

2.2. Generación de datos sintéticos

Los datos se generaron con el script generar_datos.py, que simula 500 eventos de entrega mediante np.random, cada uno con tres variables: distancia_km (distribución uniforme entre 1 y 100 km), vehiculo_tipo (variable categórica con cuatro valores posibles: camión, furgoneta, auto y bicicleta) y hora_dia (entero entre 0 y 23). El resultado se almacena en el archivo entregas.csv.

```

Primer registro generado: {'distancia_km': 38.07947176588889, 'vehiculo_tipo': np.str_('camion'), 'hora_dia': 14}
DataFrame (primeras filas):
  distancia_km  vehiculo_tipo  hora_dia
0    38.079472         camion         14
1    73.467400         camion         20
2    16.445845          auto         22
3     6.750278        bicicleta         20
4    60.510386        bicicleta          2
Dtypes:
distancia_km    float64
vehiculo_tipo    str
hora_dia         int64
dtype: object
Generados 500 eventos en entregas.csv

```

Figura 3. Ejecución del script de generación de datos sintéticos.

	distancia_km	vehiculo_tipo	hora_dia
	DOUBLE	VARCHAR	BIGINT
0	38.08	camion	14
1	73.47	camion	20
2	16.45	auto	22
3	6.75	bicicleta	20
4	60.51	bicicleta	2
5	3.04	furgoneta	23
6	83.41	furgoneta	1
7	19.00	camion	0
8	31.12	furgoneta	11
9	3.28	auto	9
10	5.62	bicicleta	15
11	24.04	furgoneta	14
12	62.22	bicicleta	22
13	98.34	camion	2
14	86.13	auto	20
15	45.60	furgoneta	3
16	94.28	furgoneta	17

Figura 4. Vista previa del dataset generado (entregas.csv).

2.3. Entrenamiento del modelo

La variable objetivo (retraso) se construyó mediante una regla determinística y simplificada: retraso = 1 si distancia_km > 60 km, y 0 en caso contrario. Esta regla asume que los envíos de larga distancia presentan una mayor probabilidad de retraso, y permite entrenar un modelo de clasificación de ejemplo sin depender de datos históricos reales.

El script entrenar_modelo.py aplica un preprocesamiento mediante ColumnTransformer, que escala las variables numéricas (distancia_km, hora_dia) con StandardScaler y codifica la variable categórica (vehiculo_tipo) con

OneHotEncoder. Ambos pasos, junto con un modelo de Regresión Logística, se integran en un único Pipeline de scikit-learn, que se entrena sobre el 80% de los datos y se evalúa sobre el 20% restante. El pipeline entrenado se serializa con joblib en models/modelo_entregas.pkl.

```
... print("Modelo guardado en models/modelo_entregas.pkl")
Datos cargados: filas= 500  columnas= 3
Primeras filas:
distancia_km vehiculo_tipo hora_dia
38.079472 camion 14
73.467400 camion 20
16.445845 auto 22
6.750278 bicicleta 20
60.510386 bicicleta 2
Dtypes:
distancia_km float64
vehiculo_tipo str
hora_dia int64
dtype: object
Distribución de la etiqueta 'retraso':
retraso
0 300
1 200
Names: count, dtype: int64
Características (X) primeras filas:
distancia_km vehiculo_tipo hora_dia
38.079472 camion 14
73.467400 camion 20
16.445845 auto 22
6.750278 bicicleta 20
60.510386 bicicleta 2
Etiqueta (y) primeros valores:
0 0
1 1
2 0
3 0
4 1
Precisión en test: 0.98
Classification report:

```

	precision	recall	f1-score	support
0	0.97	1.00	0.98	61
1	1.00	0.95	0.97	39
accuracy			0.98	100
macro avg	0.98	0.97	0.98	100
weighted avg	0.98	0.98	0.98	100

```

Confusion matrix:
[[61  0]
 [ 2 37]]
Modelo guardado en models/modelo_entregas.pkl

```

Figura 5. Ejecución exitosa del entrenamiento del modelo.

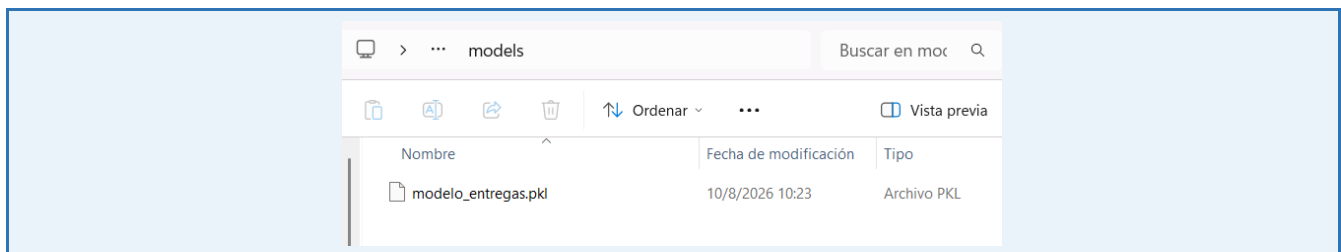


Figura 6. Evidencia de la creación del artefacto del modelo.

2.4. Despliegue de la API

El modelo entrenado se expuso como un servicio web mediante FastAPI, en el archivo app/main.py. La aplicación carga el pipeline al iniciar y expone dos endpoints: GET / (verificación de disponibilidad del servicio) y POST /predecir, que recibe distancia_km, vehiculo_tipo y hora_dia, y devuelve la probabilidad de retraso estimada por el modelo. El servicio se levantó localmente con el comando:

```
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

```

(.venv) PS C:\Users\fkva_OneDrive\Escritorio\UIDE\uide-maestria\07-despliegue-modelos\Modelo-prediccion-entregas> pip install -r requirements.txt
(.venv) PS C:\Users\fkva_OneDrive\Escritorio\UIDE\uide-maestria\07-despliegue-modelos\Modelo-prediccion-entregas> uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
INFO: Will watch for changes in these directories: ['C:\Users\fkva_OneDrive\Escritorio\UIDE\uide-maestria\07-despliegue-modelos\Modelo-prediccion-entregas']
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO: Started reload process [1304] using StatReload
C:\Users\fkva_OneDrive\Escritorio\UIDE\uide-maestria\.venv\Lib\site-packages\sklearn\base.py:463: InconsistentVersionWarning: Trying to unpickle estimator StandardScaler from vers
Use at your own risk. For more info please refer to:
https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
warnings.warn(
C:\Users\fkva_OneDrive\Escritorio\UIDE\uide-maestria\.venv\Lib\site-packages\sklearn\base.py:463: InconsistentVersionWarning: Trying to unpickle estimator OneHotEncoder from versi
Use at your own risk. For more info please refer to:
https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
warnings.warn(
C:\Users\fkva_OneDrive\Escritorio\UIDE\uide-maestria\.venv\Lib\site-packages\sklearn\base.py:463: InconsistentVersionWarning: Trying to unpickle estimator ColumnTransformer from v
ts. Use at your own risk. For more info please refer to:
https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
warnings.warn(
C:\Users\fkva_OneDrive\Escritorio\UIDE\uide-maestria\.venv\Lib\site-packages\sklearn\base.py:463: InconsistentVersionWarning: Trying to unpickle estimator LogisticRegression from
Its. Use at your own risk. For more info please refer to:
https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
warnings.warn(
C:\Users\fkva_OneDrive\Escritorio\UIDE\uide-maestria\.venv\Lib\site-packages\sklearn\base.py:463: InconsistentVersionWarning: Trying to unpickle estimator Pipeline from version 1.
t your own risk. For more info please refer to:
https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
warnings.warn(
Modelo cargado desde: C:\Users\fkva_OneDrive\Escritorio\UIDE\uide-maestria\07-despliegue-modelos\Modelo-prediccion-entregas\models\modelo_entregas.pkl
INFO: Started server process [416]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:49780 - "GET /docs HTTP/1.1" 200 OK
INFO: 127.0.0.1:49780 - "GET /openapi.json HTTP/1.1" 200 OK
Entrada recibida para predecir:
  distancia_km  vehiculo_tipo  hora_dia
      85.0      furgoneta      5
Salida de la prediccion:
{'probabilidad_retraso': 0.99}
INFO: 127.0.0.1:59376 - "POST /predecir?distancia_km=85&vehiculo_tipo=furgoneta&hora_dia=5 HTTP/1.1" 200 OK
WARNING: StatReload detected changes in 'app/main.py'. Reloading...
INFO: Shutting down
INFO: Waiting for application shutdown.
INFO: Application shutdown complete.
INFO: Finished server process [416]
C:\Users\fkva_OneDrive\Escritorio\UIDE\uide-maestria\.venv\Lib\site-packages\sklearn\base.py:463: InconsistentVersionWarning: Trying to unpickle estimator StandardScaler from vers
Use at your own risk. For more info please refer to:
https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
warnings.warn(
C:\Users\fkva_OneDrive\Escritorio\UIDE\uide-maestria\.venv\Lib\site-packages\sklearn\base.py:463: InconsistentVersionWarning: Trying to unpickle estimator OneHotEncoder from versi
Use at your own risk. For more info please refer to:
https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
warnings.warn(
C:\Users\fkva_OneDrive\Escritorio\UIDE\uide-maestria\.venv\Lib\site-packages\sklearn\base.py:463: InconsistentVersionWarning: Trying to unpickle estimator ColumnTransformer from v
ts. Use at your own risk. For more info please refer to:
https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
warnings.warn(
C:\Users\fkva_OneDrive\Escritorio\UIDE\uide-maestria\.venv\Lib\site-packages\sklearn\base.py:463: InconsistentVersionWarning: Trying to unpickle estimator LogisticRegression from

```

Figura 7. Servicio Uvicorn en ejecución con el modelo cargado.

API de Predicción de Entregas 1.0.0 OAS 3.1

/openapi.json

Predice la probabilidad de retraso en entregas logísticas.

Grupo 7:

- Azadobay Culltopala Jose Daniel
- Maldonado de la Cruz Walter Javier
- Viteri Ayala Flavia Kamila

default

GET / Root

POST /predecir Predecir

Parameters

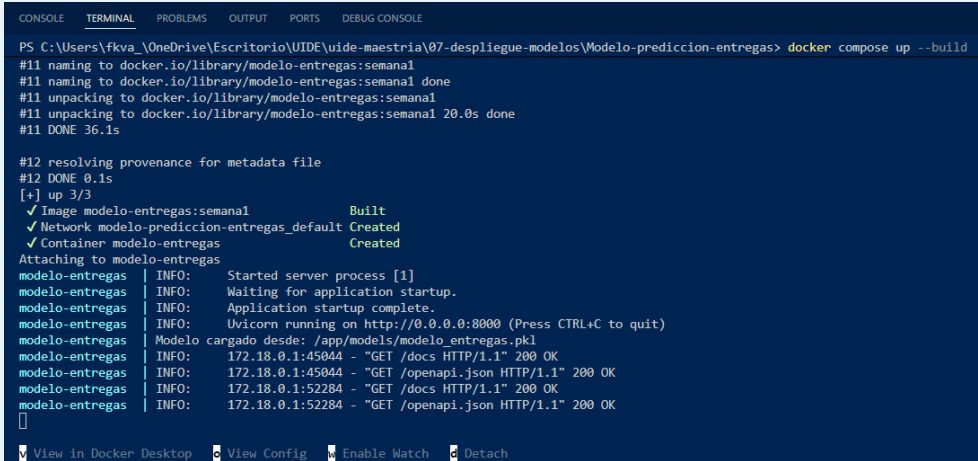
Name	Description
distancia_km * required number (query)	distancia_km
vehiculo_tipo * required string (query)	furgoneta
hora_dia * required	

Figura 8. Documentación interactiva (Swagger UI) del servicio desplegado.

2.5. Contenerización con Docker

Para garantizar la portabilidad del servicio, se empaquetó la aplicación en una imagen Docker. El Dockerfile parte de una imagen base python:3.11, copia el proyecto, instala las dependencias declaradas en requirements.txt y expone el servicio mediante Uvicorn en el puerto 8000. Adicionalmente, se definió un archivo docker-compose.yml

que monta la carpeta models como volumen, permitiendo actualizar el modelo entrenado sin necesidad de reconstruir la imagen completa.



```

PS C:\Users\fkva_OneDrive\Escritorio\UIDE\uide-maestria\07-despliegue-modelos\Modelo-prediccion-entregas> docker compose up --build
#11 naming to docker.io/library/modelo-entregas:semanal
#11 naming to docker.io/library/modelo-entregas:semanal done
#11 unpacking to docker.io/library/modelo-entregas:semanal
#11 unpacking to docker.io/library/modelo-entregas:semanal 20.0s done
#11 DONE 36.1s

#12 resolving provenance for metadata file
#12 DONE 0.1s
[+] up 3/3
✓ Image modelo-entregas:semanal Built
✓ Network modelo-prediccion-entregas_default Created
✓ Container modelo-entregas Created
Attaching to modelo-entregas
modelo-entregas | INFO: Started server process [1]
modelo-entregas | INFO: Waiting for application startup.
modelo-entregas | INFO: Application startup complete.
modelo-entregas | INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
modelo-entregas | Modelo cargado desde: /app/models/modelo_entregas.pkl
modelo-entregas | INFO: 172.18.0.1:45044 - "GET /docs HTTP/1.1" 200 OK
modelo-entregas | INFO: 172.18.0.1:45044 - "GET /openapi.json HTTP/1.1" 200 OK
modelo-entregas | INFO: 172.18.0.1:52284 - "GET /docs HTTP/1.1" 200 OK
modelo-entregas | INFO: 172.18.0.1:52284 - "GET /openapi.json HTTP/1.1" 200 OK

```

Figura 9. Construcción de la imagen Docker del servicio.

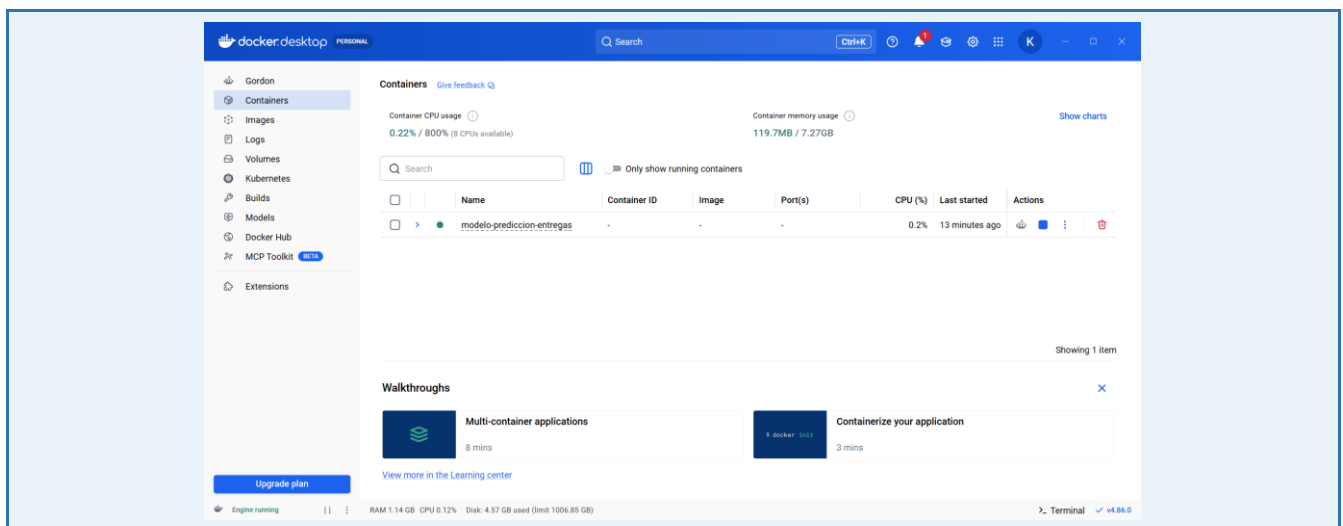


Figura 10. Contenedor Docker en ejecución.

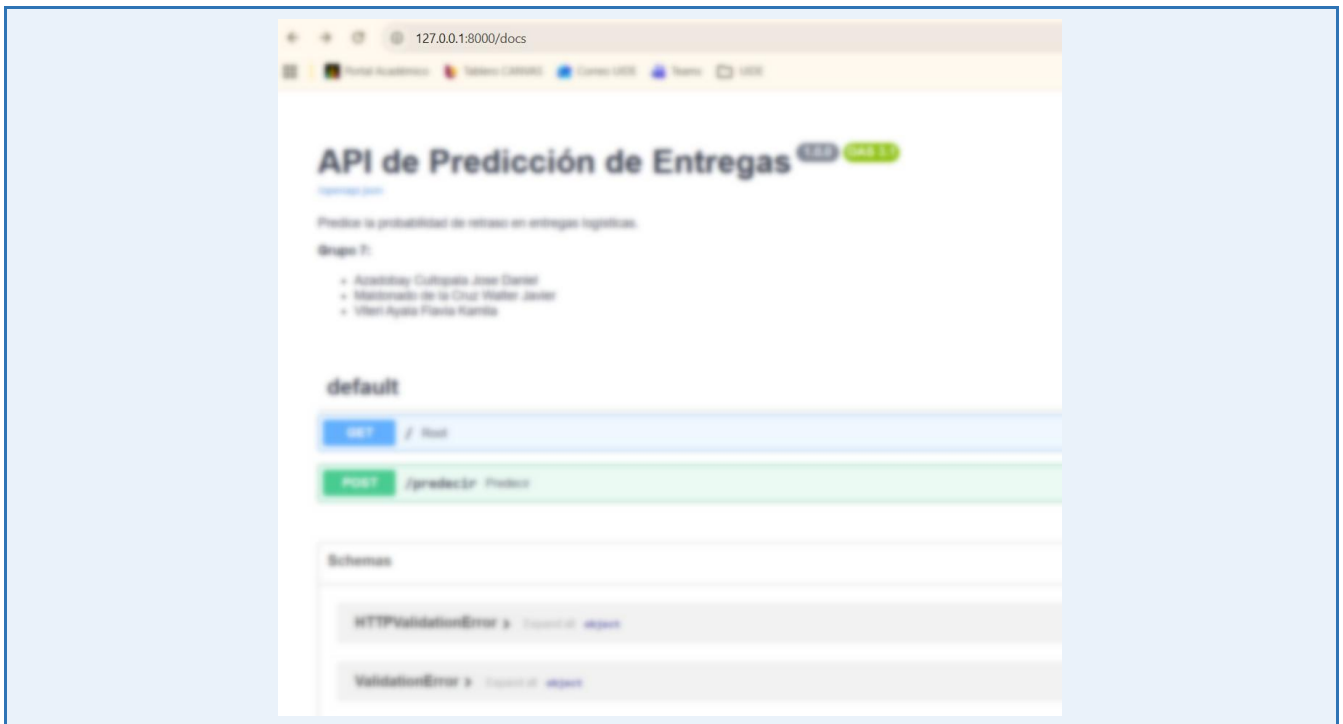


Figura 11. Verificación del servicio funcionando dentro del contenedor.

3. Resultados

Se ejecutaron dos casos de prueba sobre el endpoint `/predecir`, contrastando un escenario de distancia corta y uno de distancia larga, con el fin de validar que el modelo refleja correctamente la regla utilizada para construir la variable objetivo (Cuadro 1).

Caso	distancia_km	vehiculo_tipo	hora_dia	prob. retraso
Distancia baja	20	furgoneta	10	0
Distancia alta	85	furgoneta	5	0.99

Cuadro 1. Resultados de las pruebas de predicción para distancia baja y alta.

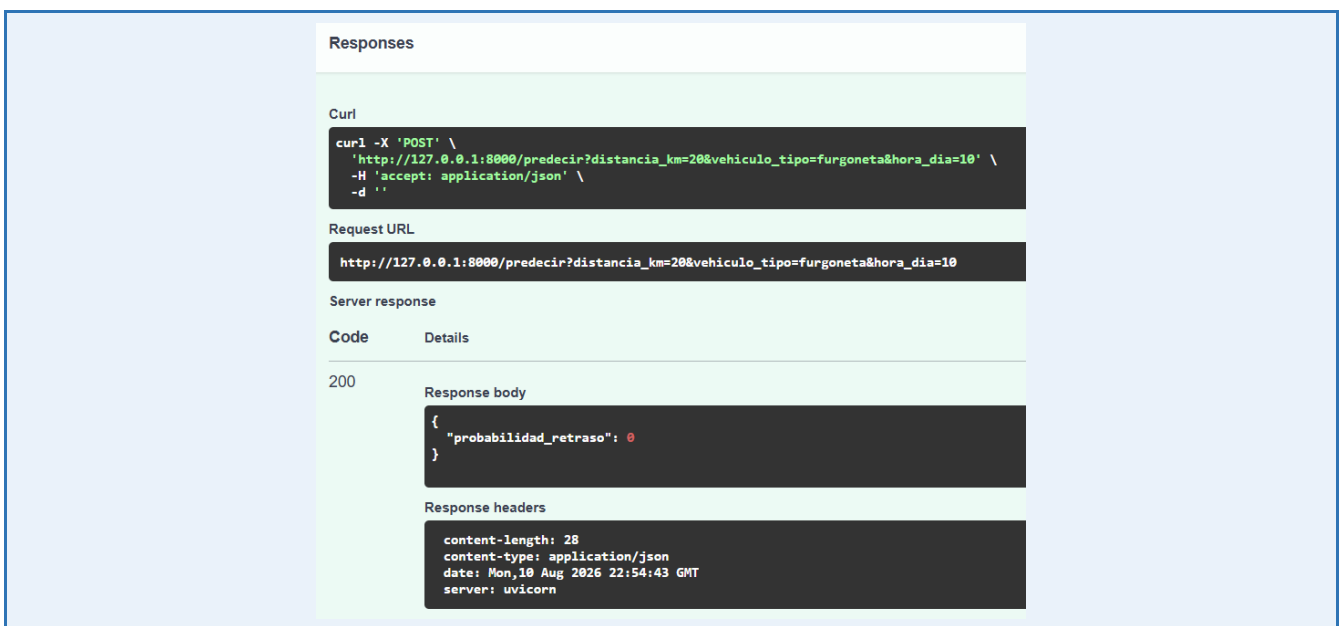


Figura 12. Respuesta de la API para el caso de distancia baja.

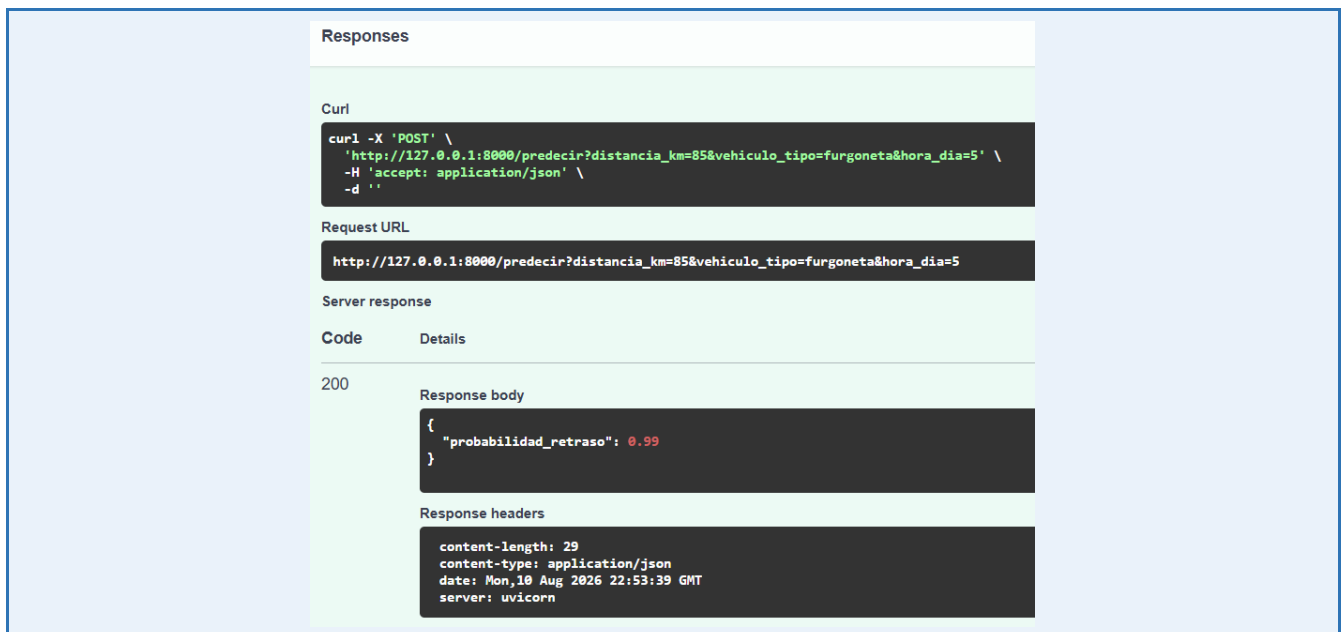


Figura 13. Respuesta de la API para el caso de distancia alta.

Dado que la variable objetivo se definió con la regla `distancia_km > 60`, se espera que el modelo asigne una probabilidad cercana a 0 para el caso de distancia baja y cercana a 1 para el caso de distancia alta, reflejando una frontera de decisión definida en torno a este rango.

4. Mejoras Aplicadas al Entregable

Una vez ejecutado el código base, identificaron dos oportunidades de mejora relacionadas con la reproducibilidad de los datos y la robustez de la API frente a entradas inválidas. Ambas fueron implementadas y se documentan a continuación siguiendo el formato estado anterior – cambio aplicado – resultado.

4.1. Reproducibilidad: fijación de semilla aleatoria

Estado anterior: el script `generar_datos.py` no fijaba una semilla para el generador de números aleatorios de NumPy, por lo que cada ejecución producía un conjunto de datos distinto (`distancia_km`, `vehiculo_tipo` y `hora_dia` variaban entre corridas), impidiendo que en cada corrida se obtuvieran resultados comparables.

Cambio aplicado: se incorporó `np.random.seed(42)` antes de generar los 500 eventos sintéticos, fijando el estado interno del generador aleatorio.

Resultado / Influencia: con este cambio, con cualquier ejecución de `generar_datos.py` se obtiene exactamente el mismo archivo `entregas.csv`, garantizando que el modelo entrenado posteriormente sea reproducible, en línea con las prácticas de trazabilidad y reproducibilidad discutidas en el marco de MLOps.

4.2. Robustez de la API: validación de vehiculo_tipo

Estado anterior: el parámetro `vehiculo_tipo` del endpoint POST `/predecir` estaba tipado como una cadena de texto genérica (`str`), sin restricción sobre los valores aceptados. Al enviar una categoría no contemplada en el entrenamiento del modelo (por ejemplo, "moto"), el servicio devolvía un error no controlado (HTTP 500), generado por el OneHotEncoder al intentar codificar una categoría desconocida.

Cambio aplicado: se redefinió el tipo del parámetro utilizando `Literal["camion", "furgoneta", "auto", "bicicleta"]`, restringiendo explícitamente los valores aceptados por la API a las categorías presentes en los datos de entrenamiento.

Resultado / Influencia: la interfaz de Swagger UI transformó automáticamente el campo de texto libre en un menú desplegable, y cualquier valor fuera de las cuatro categorías válidas es ahora rechazado con un error de validación controlado (HTTP 422) en lugar de un error interno del servidor. Este cambio conecta directamente con las

prácticas de validación de entradas mediante esquemas estructurados discutidas en la teoría de seguridad de APIs en producción.

The screenshot shows a Swagger UI for a POST endpoint named `/predecir` with the description 'Predecir'. Under the 'Parameters' section, there are three required parameters:

- `distancia_km`: A number (query) with a value of 20.
- `vehiculo_tipo`: A string (query) represented as a dropdown menu. The menu is open, showing options: `furgoneta`, `camion`, `furgoneta` (highlighted), `auto`, and `bicicleta`.
- `hora_dia`: An integer (query).

An 'Execute' button is located at the bottom of the parameter section.

Figura 14. Campo `vehiculo_tipo` convertido en menú desplegable mediante Literal.

5. Conclusiones

- ¿Qué parte del ejercicio fue más clara?

La parte que resultó más clara para nosotros fue el flujo de entrenamiento del modelo con scikit-learn, particularmente su empaquetado dentro de un Pipeline. Entendimos con facilidad por qué tiene sentido integrar el preprocesamiento (el `ColumnTransformer` con `StandardScaler` para las variables numéricas y `OneHotEncoder` para la categórica) junto con el clasificador en un mismo objeto, de esta manera, cualquier dato nuevo que llegue a la API pasa automáticamente por las mismas transformaciones que se aplicaron durante el entrenamiento, sin que nosotros tengamos que recordar manualmente el orden de los pasos ni el riesgo de aplicar un escalado distinto al que aprendió el modelo.

De igual forma, la lógica de exposición del modelo mediante FastAPI se comprendió sin mayor dificultad una vez identificamos el patrón general, el endpoint `POST /predecir` simplemente recibe los mismos tipos de datos con los que se entrenó el pipeline, arma un `DataFrame` con esa información y delega en el modelo ya serializado toda la responsabilidad de transformar y predecir. Ver que `joblib.load()` reconstruye el pipeline completo (preprocesamiento incluido) con una sola línea de código fue un punto de claridad importante, porque conecta directamente con lo revisado en la teoría sobre por qué se recomienda empaquetar el modelo como una unidad reproducible antes de llevarlo a producción.

Por último, el uso de la documentación automática de Swagger UI facilitó bastante la validación de que el servicio funcionaba como se esperaba, sin necesidad de escribir código adicional para probar el endpoint. Esto nos permitió concentrarnos en verificar la lógica de negocio (que la probabilidad de retraso respondiera correctamente al umbral de distancia definido) en lugar de invertir tiempo en construir una interfaz de prueba desde cero.

- ¿Qué parte fue más difícil?

La mayor dificultad del ejercicio no estuvo en el código del modelo ni en la lógica de la API, sino en la gestión del entorno de ejecución alrededor de ellos. Cometimos en repetidas ocasiones el error de ejecutar comandos (ya fuera `pip install`, `uvicorn` o `docker compose`) desde la raíz del repositorio general de la materia en lugar de la carpeta específica del proyecto, lo cual generaba errores como `ModuleNotFoundError: No module named 'app'` o `FileNotFoundError` al buscar `requirements.txt`. Estos errores resultaron confusos al principio porque el mensaje no indicaba directamente "estás en la carpeta equivocada", sino que apuntaba a síntomas (un módulo no encontrado, un archivo inexistente) cuya causa raíz solo se hizo evidente al revisar con cuidado la ruta mostrada en el propio prompt de la terminal.

También tuvimos que resolver, ya en la fase de Docker, que la máquina no tenía habilitada la virtualización a nivel de BIOS, lo cual impedía que Docker Desktop iniciara su motor, un obstáculo completamente ajeno al contenido de la materia, pero que demuestra que el despliegue de un modelo depende de una cadena de prerrequisitos de infraestructura que van mucho más allá de escribir el código correcto.

En retrospectiva, consideramos que estas dificultades nos ayudaron a desarrollar el hábito de verificar sistemáticamente en qué directorio estamos, si el entorno virtual está activo, y si los servicios de infraestructura (como el motor de Docker) están efectivamente corriendo, antes de asumir que un error proviene del código. Esta distinción entre error de aplicación y error de entorno es, a nuestro parecer, una de las competencias prácticas más relevantes que deja este ejercicio para un futuro rol en despliegue de modelos.

- ¿Qué utilidad ven en desplegar un modelo de IA como servicio?

El ejercicio nos permitió comprobar de manera práctica una idea que hasta entonces solo habíamos abordado desde la teoría, ya que un modelo entrenado no genera valor por sí mismo hasta que puede ser consumido por otros sistemas (o personas) sin que estos necesiten conocer Python, scikit-learn ni los detalles internos del pipeline que lo compone. Al exponer el modelo mediante una API REST, este se convierte en un componente reutilizable dentro de una arquitectura más amplia (tal como se plantea en el caso de la empresa de logística simulada en este caso), donde una aplicación de seguimiento de entregas, un sistema de asignación de rutas o incluso un dashboard interno podrían consultar la probabilidad de retraso con una simple petición HTTP, sin depender de que el equipo de ciencia de datos les entregue el modelo directamente.

Asimismo, la contenerización con Docker reforzó nuestra comprensión de por qué la portabilidad es un requisito central del despliegue: verificar que el mismo servicio respondiera de forma idéntica dentro del contenedor y fuera de él nos dio evidencia concreta de que el modelo ya no depende del entorno particular de la máquina de un integrante del equipo, sino que puede moverse a cualquier servidor o proveedor de nube que soporte Docker, exactamente como se planteó en la teoría de MLOps y arquitectura de sistemas de IA en producción.

Por último, el hallazgo del error HTTP 500 al enviar una categoría de vehículo no contemplada en el entrenamiento (documentado y corregido en la Sección 4.2 mediante la restricción de tipos con Literal) fue quizás la lección más concreta sobre la importancia de desplegar con criterio y no solo con funcionalidad. Nos permitió dimensionar, con un caso real y no hipotético, por qué la validación de entradas no es un detalle secundario, debido a que un servicio que falla de forma no controlada frente a un dato inesperado no puede considerarse verdaderamente listo para producción, sin importar qué tan preciso sea el modelo que tiene detrás.

Anexos

Enlace al repositorio del proyecto: [GitHub repository](#)

Código fuente completo: `main.py`, `entrenar_modelo.py`, `generar_datos.py`, `Dockerfile`, `docker-compose.yml`, `requirements.txt`.

Respuestas al Test Semana 1: Despliegue de modelos de IA en Producción